

Kalman Filtering III: Linear-Quadratic-Gaussian (LQG) Control

LQR optimal control + Kalman optimal estimation = LQG.

When the state is not measured **and** the plant is driven by noise, the **separation principle** still holds: design the LQR feedback and the Kalman filter independently, then join them. The result is the LQG compensator. In this tutorial you will:

- design the LQR gain and the Kalman gain separately,
- assemble the observer-based LQG compensator, and
- regulate a noisy, partially measured plant (before/after).

Combines `State-Space/QuadraticOptimalRegulator.m` (LQR) with the Kalman filter, and mirrors the structure of `State-Space/RegulatorSystems.m`. Run with `publish('LQG.m')`.

Contents

- Plant with process and measurement noise
- Step 1 – LQR feedback (the control half)
- Step 2 – Kalman filter (the estimation half)
- Step 3 – Assemble the LQG compensator
- Separation principle under noise
- Before vs. after: open loop vs. LQG on the noisy plant
- Try it yourself

Plant with process and measurement noise

A double integrator: process noise w enters through G , and sensor noise v corrupts the single measured output. Left alone, a noise-driven double integrator drifts away.

```
A = [0 1; 0 0];
B = [0; 1];
C = [1 0];
G = [0; 1];           % process-noise input
Qn = 1;               % process-noise intensity
Rn = 0.1;             % measurement-noise intensity
```

Step 1 – LQR feedback (the control half)

Weight state and control; `lqr` returns the optimal $u = -K_c x$.

```
Q = diag([10 1]); Rc = 1;
Kc = lqr(A,B,Q,Rc);
fprintf('LQR gain Kc = %s\n', mat2str(Kc,4))
```

```
LQR gain Kc = [3.162 2.706]
```

Step 2 – Kalman filter (the estimation half)

Solve the filter Riccati equation for the optimal estimator gain K_f . (`lqe` and `kalman` are the one-line toolbox equivalents.)

```
Pf = care(A', C', G*Qn*G', Rn);
Kf = Pf*C'/Rn;
fprintf('Kalman gain Kf = %s\n', mat2str(Kf',4))
```

```
Kalman gain Kf = [2.515 3.162]
```

Step 3 – Assemble the LQG compensator

$u = -K_c \hat{x}$ with $\dot{\hat{x}} = (A - BK_c - K_f C)\hat{x} + K_f y$ – the same observer-based structure as `RegulatorSystems.m`, now with the **optimal** control and estimation gains. (`lqgreg` builds this from a `kalman` estimator and K_c in one call.)

```
Acmp = A - B*Kc - Kf*C;
comp = ss(Acmp, Kf, -Kc, 0);          % input y, output u
```

Separation principle under noise

The closed-loop poles are exactly the LQR poles union the Kalman poles – the two designs never interfere.

```
Az = [A, -B*Kc; Kf*C, Acmp];          % closed loop in [x; xhat] coordinates
fprintf('\nClosed-loop poles: %s\n', mat2str(sort(eig(Az)).',3))
fprintf('LQR poles:          %s\n', mat2str(sort(eig(A-B*Kc)).',3))
fprintf('Kalman poles:       %s\n', mat2str(sort(eig(A-Kf*C)).',3))
```

```
Closed-loop poles: [-1.26-1.26i -1.26+1.26i -1.35-1.15i -1.35+1.15i]
LQR poles:         [-1.35-1.15i -1.35+1.15i]
Kalman poles:      [-1.26-1.26i -1.26+1.26i]
```

Before vs. after: open loop vs. LQG on the noisy plant

Drive the true plant with process noise from a nonzero initial state. Open loop it drifts; the LQG compensator – seeing only the noisy output y – pulls it back toward zero.

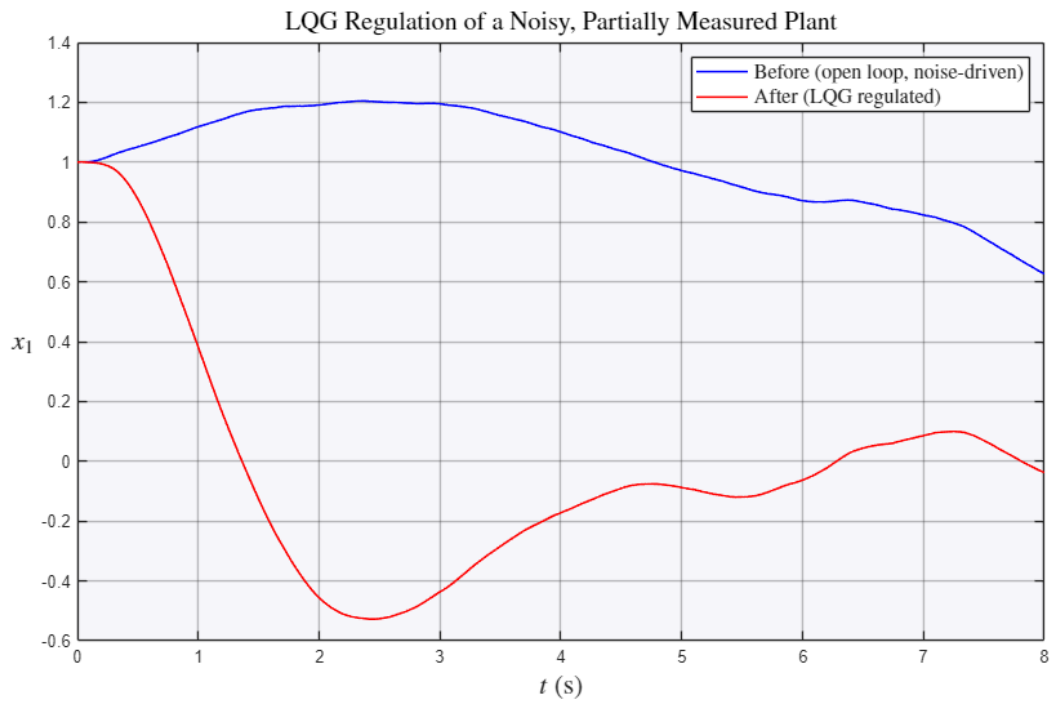
```
rng(0);
t = 0:0.01:8; N = numel(t);
w = sqrt(Qn)*randn(1,N);          % process noise
v = sqrt(Rn)*randn(1,N);          % sensor noise
x0 = [1; 0];

y_ol = lsim(ss(A,G,C,0), w, t, x0); % open loop (u = 0)

Bz = [G, zeros(2,1); zeros(2,1), Kf]; % [w; v] -> [x; xhat]
sys_cl = ss(Az, Bz, [1 0 0 0], 0);    % output = true x1
x1_cl = lsim(sys_cl, [w; v].', t, [x0; 0; 0]); % inputs as columns: N-by-2

figure
plot(t, y_ol, 'b', t, x1_cl, 'r', 'LineWidth', 1.2)
grid on
legend('Before (open loop, noise-driven)', 'After (LQG regulated)', 'Interpreter', 'latex', 'FontSize', 12)
title('LQG Regulation of a Noisy, Partially Measured Plant', 'Interpreter', 'latex', 'FontSize', 15)
ylabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 16); set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$ (s)', 'Interpreter', 'latex', 'FontSize', 16)
fprintf('\nRMS x1 -- open loop: %.3f, LQG: %.3f\n', ...
    sqrt(mean(y_ol.^2)), sqrt(mean(x1_cl.^2)))
```

```
RMS x1 -- open loop: 1.027, LQG: 0.377
```



Try it yourself

- Raise the sensor-noise intensity R_n and notice K_f shrink (trust the model more): the estimate gets smoother but laggier.
- Increase the LQR state weight Q and watch the regulation tighten – the control and filter tune independently, exactly as separation promises.